

資料操作 Data Manipulating

Computer Architecture

● CPU

- CU(Control Unit)：負責指揮指令與資料在 CPU 和記憶體間的流動

(抓取指令、解碼、發送控制信號指揮下一步、更新 Program Counter)

- ALU(Arithmetic Logic Unit)：負責邏輯判斷與數學運算

- Register

- ◆ General purpose register：不限用途，由程式決定內容放什麼

- ◆ Special purpose：與 CPU 控制狀態有關，固定功能不可任意修改

- ◇ Program Counter(PC)：記錄下一條要執行的指令位址

- ◇ Instruction Register(IR)：存放目前正在執行的指令

- Bus：電腦各部件之間傳輸資料的通道。CPU 與 Memory 之間有一個取指令、存取資料的 Bus

- Main Memory (RAM)

以算數學題目比較CPU暫存器、RAM、硬碟

CPU暫存器:腦袋閃過的公式, 最快, 但容易忘記也記不久

RAM:腦袋記不起來的暫時寫在計算紙上

硬碟:算完答案寫上答案卡

CISC vs RISC

CISC (複雜指令集)	RISC (精簡指令集)
指令集龐大且複雜	指令集較少且簡單
指令長度不固定	指令長度固定，易於解碼
功能強大、執行時間長、功耗較大	功能精簡、效率較高、功耗較低
x86(Intel. AMD)	ARM

Controller

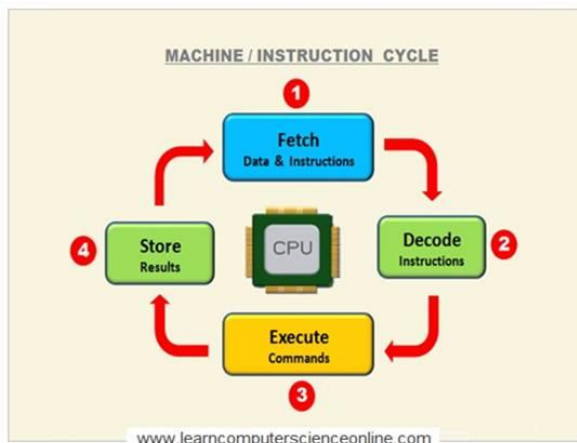
處理電腦和其他裝置之間訊息溝通的中介裝置

***與 CU 不同，不可混淆**

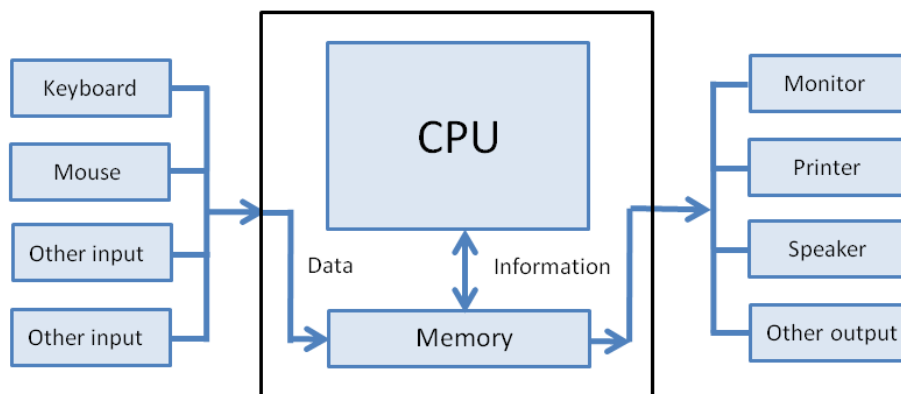
Stored-Program Concept

*早期電腦是固定式架構，改程式要手動重接電路。Stored Program 則是把程式存在 Main Memory

- 換程式就能改變電腦功能
- Machine Cycle = Instruction Cycle
有時候會把 Store 包含在 Execute，也就是 Fetch-Decode-Execute Cycle



- von Neumann architecture 馮·諾伊曼架構(補，可惡的臺師大資工面試 rrrr)



指令和資料共用同一個記憶體空間

→ CPU 與記憶體競爭匯流排，造成 CPU 要等較慢的記憶體或 IO 傳輸指令或資料而閒置
(馮·諾伊曼瓶頸 Von Neumann Bottleneck)

DMA

Direct Memory Access

- I/O 設備可以直接存取 Memory，不需要透過 CPU 傳輸
- CPU 只需初始化 DMA 控制器，之後的傳輸過程由 **DMA 控制器** 獨立完成
- 緩解 CPU 負載
- 但也造成 Bus 更大的負載，新的瓶頸

◆ 總結對比表				
面向	有無 DMA	對 CPU 的影響	對匯流排的影響	瓶頸是否緩解？
傳統 I/O	無 DMA	CPU 忙於搬資料，效率低	匯流排負載低（CPU 串行使用）	CPU 瓶頸嚴重
使用 DMA	有 DMA	CPU 可並行做事，效率高	匯流排負載高（DMA 與 CPU 爭用）	CPU 瓶頸緩解，但匯流排瓶頸仍在

<https://vocus.cc/article/67ea9269fd89780001ab4c6f>

💡 運作流程

傳統 I/O 傳輸（沒有 DMA）：

1. I/O 裝置 → CPU → 記憶體，一筆一筆搬。
2. CPU 忙於資料搬移，幾乎不能做其他工作。

使用 DMA：

1. CPU 先設定 DMA 控制器（來源位址、目的位址、資料長度）。
2. DMA 控制器向 CPU 請求控制匯流排（Bus）。
3. 獲准後，DMA 控制器直接在 記憶體與 I/O 裝置之間 傳資料。
4. 傳完再以中斷（Interrupt）通知 CPU。

Handshaking

Handshaking 是一種在兩個裝置之間進行的「訊號協調機制」，確保資料傳輸在正確時機進行，避免時序衝突。

- I/O 裝置送 Ready
- CPU 回覆 Acknowledge

並列埠 vs 序列埠

	並列埠(Parallel)	序列埠(Serial)
傳輸方式	一次傳送多個位元	一次傳送 1 個位元，逐位傳輸
資料線路數	多條	少條 (TX、RX、GND)
傳輸速度	理論上快	理論上慢
成本	高，很多根	低
穩定性	長距離傳輸易被干擾 且有 訊號同步問題	穩定性高，抗干擾強 慢但可靠

USB 透過現代線路設計，序列傳輸也很快。現在幾乎沒有並列埠了

Clock Cycle

CPU 的最小時間單位，與時脈頻率為倒數

一顆 3 GHz 的 CPU $\Rightarrow$ 一個 clock cycle 約為

$$\frac{1}{3 \times 10^9} = 0.33\text{ns}$$

每一次 clock tick，CPU 可以執行一個最基本的動作（例如傳輸一個位元、做一次暫存器操作）。

一個 Machine Cycle 可能需要多個 Clock Cycle

Machine Language

✧ Machine Instruction

- CPU 能直接識別並執行的最基本指令，與組合語言對應
- 以 2 進位表示
- Opcode：告訴 CPU 要執行什麼動作（例如 ADD、LOAD、JUMP）
- Operand：指定要操作的資料或位置（例如寄存器、記憶體位址、常數）

🧠 範例（概念上）：

```
yaml
```

Opcode	Operand
LOAD	R1, 1000

意思是「將記憶體位址 1000 的值載入寄存器 R1」。

✧ Unconditional Jump（以課本為例）

JUMP 比較「指定 Register」和「Register 0」，如果兩者 content 相等就 JUMP，因此指定 Register0 就可以自己比自己，必定會執行。

Python Operator

&: AND

||: OR

^: XOR $\text{bin}(0b1011 \wedge 0b1101) \rightarrow (0b0110)$

Functions

✧ Fruitful function：有 return value，會回傳一個結果

✧ Procedure(void function)：不會 return value，用來執行（print、更新資料、上傳檔案...）

Rotation & Shift

Logical Shift：把 Bit pattern 左移或右移，空位補 0

Arithmetic Shift：把 Bit pattern 左移或右移，空位補 sign bit 維持正負號

Circular shift = Rotation：Bit pattern 左移或右移後，溢出的位數往另一側補

Pipelining

重疊 Machine Cycle，同時處理不同指令的不同執行階段，就像工廠流水線工位一樣。

傳統 CPU 執行一條指令通常要經歷以下步驟（以典型 5 階段為例）：

1. IF (Instruction Fetch)：從記憶體取出指令
2. ID (Instruction Decode / Register Fetch)：解碼指令、讀取暫存器
3. EX (Execute / ALU Operation)：執行運算或地址計算
4. MEM (Memory Access)：讀寫記憶體（若指令需要）
5. WB (Write Back)：將結果寫回暫存器

傳統（非流水線）CPU 一次只能處理一條指令的完整五個階段。

傳統 CPU 一次只能處理一條指令的所有階段。

當第一條指令進入 ID 階段時，第二條指令可以開始 IF 階段.....

Parallel Processing 平行運算

同時用多個 Processor 運算，以 Instruction Stream 指令流和 Data Stream 資料流分

✧ SISD：Single Instruction, Single Data 沒有平行運算

✧ MIMD：Multiple Instruction, Multiple Data 一般多核心 CPU 或分散式系統，多程式並行

✧ SIMD：Single Instruction, Multiple Data 一組指令同時運算多組資料